

```
"""
evaluate.py - full test-set evaluation, per-case diagnostics, and plots.
```

Usage

```
python src/evaluate.py \
    --data    data/physics_train.csv \
    --ckpt    checkpoints/best.pt \
    --norms    checkpoints/norm_stats.npz \
    --out     outputs/
```

Produces

```
outputs/test_predictions.csv - predicted vs true per timestep
outputs/test_case_metrics.csv - per-case RMS error and correlation
outputs/plots/case_*.png      - time-series comparison plots
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
import sys
```

```
from pathlib import Path
```

```
from typing import Dict, List
```

```
import matplotlib
```

```
matplotlib.use("Agg")          # headless
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import pandas as pd
```

```
import torch
```

```
sys.path.insert(0, str(Path(__file__).parent))
```

```
from config import BASE_CFG, DT, STATE_NAMES, T_IGNORE, TRAIN_CFG
```

```
from dataset import ACCEL_COLS, CabinDataset, NormStats
```

```
from model import PhysicsLSTM
```

```
from physics import precompute_road_array
```

```
# -----
# Prediction helper
# -----
```

```

@torch.no_grad()
def predict_case(model: PhysicsLSTM,
                 sample: Dict,
                 ns: NormStats,
                 device: str) -> Dict:
    """
    Run model on one sample.
    Returns dict of numpy arrays (physical units).
    """
    road = sample["road"].unsqueeze(0).to(device)
    param = sample["param"].unsqueeze(0).to(device)

    sp, ap = model(road, param)

    # Denormalise
    s_pred = (sp[0].cpu().numpy() * ns.s_std + ns.s_mean) # [T, 6]
    a_pred = (ap[0].cpu().numpy() * ns.a_std + ns.a_mean) # [T, 3]
    s_true = (sample["state"].numpy() * ns.s_std + ns.s_mean)
    a_true = (sample["accel"].numpy() * ns.a_std + ns.a_mean)
    rms_true = float(sample["rms"].numpy()[0])

    return {
        "s_pred": s_pred, "a_pred": a_pred,
        "s_true": s_true, "a_true": a_true,
        "rms_true": rms_true,
    }

def seat_rms(a_arr: np.ndarray, h_seat: float = 0.1,
             t_arr: np.ndarray = None, t_ignore: float = T_IGNORE) -> float:
    """
    ISO 2631-style combined 3-axis seat acceleration RMS.
    a_arr: [T, 3] - accels (qdd_z_c, qdd_th_c, qdd_ph_c) in physical units.
    h_seat used as hcp lever arm.
    """
    if t_arr is not None:
        mask = t_arr >= t_ignore
        a_arr = a_arr[mask]
    if len(a_arr) == 0:
        return float("nan")
    az = a_arr[:, 0]
    ax = -h_seat * a_arr[:, 1]
    ay = h_seat * a_arr[:, 2]
    return float(np.sqrt(np.mean(az**2) + np.mean(ax**2) + np.mean(ay**2)))

```

```

# -----
# Plot one case
# -----

def plot_case(case_id: int,
              t: np.ndarray,
              pred: Dict,
              out_path: Path,
              h_seat: float = 0.6) -> None:
    fig, axes = plt.subplots(4, 1, figsize=(14, 10), sharex=True)
    fig.suptitle(f"Case {case_id}", fontsize=12)

    names = STATE_NAMES[:3]
    labels = ["Bounce z_c [m]", "Pitch  $\theta_c$  [rad]", "Roll  $\phi_c$  [rad]"]
    for i, (ax, lbl) in enumerate(zip(axes[:3], labels)):
        ax.plot(t, pred["s_true"][:, i], "k-", lw=0.8, label="ODE")
        ax.plot(t, pred["s_pred"][:, i], "r--", lw=0.8, label="Surrogate")
        ax.set_ylabel(lbl, fontsize=9)
        ax.legend(fontsize=8, loc="upper right")
        ax.grid(True, lw=0.4)

    # Seat accel
    h = h_seat # hcp lever arm
    az_t = pred["a_true"][:, 0]; ax_t = -h*pred["a_true"][:, 1]; ay_t = h*pred["a
    az_p = pred["a_pred"][:, 0]; ax_p = -h*pred["a_pred"][:, 1]; ay_p = h*pred["a
    a_seat_true = np.sqrt(az_t**2 + ax_t**2 + ay_t**2) # instantaneous combined
    a_seat_pred = np.sqrt(az_p**2 + ax_p**2 + ay_p**2)
    rms_t = seat_rms(pred["a_true"], h_seat)
    rms_p = seat_rms(pred["a_pred"], h_seat)
    axes[3].plot(t, a_seat_true, "k-", lw=0.8,
                 label=f"ODE RMS={rms_t:.4f}")
    axes[3].plot(t, a_seat_pred, "r--", lw=0.8,
                 label=f"Surr RMS={rms_p:.4f}")
    axes[3].set_ylabel("Seat accel [m/s2]", fontsize=9)
    axes[3].set_xlabel("Time [s]", fontsize=9)
    axes[3].legend(fontsize=8, loc="upper right")
    axes[3].grid(True, lw=0.4)

    plt.tight_layout()
    fig.savefig(str(out_path / f"case_{case_id:04d}.png"), dpi=120)
    plt.close(fig)

# -----
# Full evaluation
# -----

```

```

def evaluate(
    data_csv: str,
    ckpt_path: str,
    norms_path: str,
    out_dir: str,
    cfg_base: Dict = None,
    train_cfg: Dict = None,
    n_plot: int = 10,      # max cases to plot
    split: str = "test",
) -> pd.DataFrame:

    cfg_base = cfg_base or BASE_CFG
    train_cfg = train_cfg or TRAIN_CFG
    out_path = Path(out_dir); out_path.mkdir(parents=True, exist_ok=True)
    plot_path = out_path / "plots"; plot_path.mkdir(exist_ok=True)

    device = "cuda" if torch.cuda.is_available() else "cpu"
    h_seat = float(cfg_base.get("hcp", 0.1)) # ISO 2631: use hcp as lever arm
    ds = int(train_cfg.get("downsample", 4))

    # — load norm stats —————
    ns = NormStats.load(norms_path)

    # — load model —————
    model = PhysicsLSTM(
        road_dim=train_cfg["road_dim"],
        param_dim=train_cfg["param_dim"],
        lstm_hidden=train_cfg["lstm_hidden"],
        lstm_layers=train_cfg["lstm_layers"],
        dropout=0.0, # no dropout at eval
    ).to(device)
    ckpt = torch.load(ckpt_path, map_location=device)
    model.load_state_dict(ckpt["model"])
    model.eval()
    print(f"Loaded model from {ckpt_path} (epoch {ckpt.get('epoch', '?')})")

    # — pre-compute road —————
    t_eval = np.arange(0.0, float(cfg_base["sim_duration_s"]) + DT, DT)[::ds]
    road_arr = precompute_road_array(cfg_base, t_eval)

    # — dataset —————
    ds_obj = CabinDataset(data_csv, ns, split=split,
                           h_seat=h_seat, road_arr=road_arr)
    print(f"Evaluating {len(ds_obj)} {split} cases ...")

    # — per-case evaluation —————
    case_metrics = []

```

```

pred_rows    = []

for i, sample in enumerate(ds_obj):
    cid = sample["case_id"]
    pred = predict_case(model, sample, ns, device)

    T = pred["s_pred"].shape[0]
    t_arr = t_eval[:T]

    rms_pred = seat_rms(pred["a_pred"], h_seat)
    rms_true = seat_rms(pred["a_true"], h_seat)
    rms_err = abs(rms_pred - rms_true) / (rms_true + 1e-8)

    # State MAE (z_c)
    mae_zc = float(np.mean(np.abs(pred["s_pred"][:, 0] - pred["s_true"][:, 0])

    # Pearson correlation on seat accel
    h_c = float(cfg_base.get("hcp", 0.1))
    az_t2 = pred["a_true"][:, 0]; ax_t2 = -h_c*pred["a_true"][:, 1]; ay_t2 =
a_s_t = np.sqrt(az_t2**2 + ax_t2**2 + ay_t2**2)
    az_p2 = pred["a_pred"][:, 0]; ax_p2 = -h_c*pred["a_pred"][:, 1]; ay_p2 =
a_s_p = np.sqrt(az_p2**2 + ax_p2**2 + ay_p2**2)
    if a_s_t.std() > 1e-8 and a_s_p.std() > 1e-8:
        corr = float(np.corrcoef(a_s_t, a_s_p)[0, 1])
    else:
        corr = float("nan")

    case_metrics.append({
        "case_id": cid,
        "rms_true": rms_true,
        "rms_pred": rms_pred,
        "rms_rel_err": rms_err,
        "mae_zc": mae_zc,
        "corr_aseat": corr,
    })

# Store predictions
for k in range(T):
    row = {"case_id": cid, "t": float(t_arr[k])}
    for j, n in enumerate(STATE_NAMES[:3]):
        row[f"{n}_pred"] = float(pred["s_pred"][k, j])
        row[f"{n}_true"] = float(pred["s_true"][k, j])
    for j, n in enumerate(["qdd_z_c", "qdd_th_c", "qdd_ph_c"]):
        row[f"{n}_pred"] = float(pred["a_pred"][k, j])
        row[f"{n}_true"] = float(pred["a_true"][k, j])
    pred_rows.append(row)

```

```

    # Plot
    if i < n_plot:
        plot_case(cid, t_arr, pred, plot_path, h_seat)

# — summary —————
cm_df = pd.DataFrame(case_metrics)
cm_df.to_csv(str(out_path / "test_case_metrics.csv"), index=False)

pred_df = pd.DataFrame(pred_rows)
pred_df.to_csv(str(out_path / "test_predictions.csv"), index=False)

print("\n=== Test set summary ===")
print(f"  Cases evaluated      : {len(cm_df)}")
print(f"  RMS rel. error mean : {cm_df['rms_rel_err'].mean()*100:.2f}%")
print(f"  RMS rel. error max  : {cm_df['rms_rel_err'].max()*100:.2f}%")
print(f"  Seat accel corr.    : {cm_df['corr_aseat'].mean():.4f}")
print(f"  MAE z_c [m]         : {cm_df['mae_zc'].mean():.6f}")

summary = {
    "n_cases": len(cm_df),
    "rms_rel_err_mean": float(cm_df["rms_rel_err"].mean()),
    "rms_rel_err_max": float(cm_df["rms_rel_err"].max()),
    "corr_aseat_mean": float(cm_df["corr_aseat"].mean()),
    "mae_zc_mean": float(cm_df["mae_zc"].mean()),
}

with open(str(out_path / "test_summary.json"), "w") as f:
    json.dump(summary, f, indent=2)

# — RMS parity plot —————
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(cm_df["rms_true"], cm_df["rms_pred"],
           s=40, alpha=0.7, edgecolors="k", linewidths=0.5)
lim = max(cm_df["rms_true"].max(), cm_df["rms_pred"].max()) * 1.05
ax.plot([0, lim], [0, lim], "k--", lw=1, label="Perfect")
ax.set_xlabel("ODE seat RMS [m/s2]")
ax.set_ylabel("Surrogate seat RMS [m/s2]")
ax.set_title("Parity plot - cabin seat RMS")
ax.legend(); ax.grid(True, lw=0.4)
plt.tight_layout()
fig.savefig(str(out_path / "rms_parity.png"), dpi=150)
plt.close(fig)
print(f"\nParity plot → {out_path / 'rms_parity.png'}")
print(f"Case metrics → {out_path / 'test_case_metrics.csv'}")

return cm_df

```

```

# -----
# CLI
# -----

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Evaluate surrogate on test set")
    parser.add_argument("--data", required=True)
    parser.add_argument("--ckpt", required=True)
    parser.add_argument("--norms", required=True)
    parser.add_argument("--out", default="outputs/eval")
    parser.add_argument("--split", default="test",
                        choices=["train", "test", "all"])
    parser.add_argument("--n_plot", type=int, default=10,
                        help="Max cases to plot")
    parser.add_argument("--data_dir", type=str, default=None)
    args = parser.parse_args()

    cfg = dict(BASE_CFG)
    if args.data_dir:
        for side in ("front_left", "front_right",
                    "rear1_left", "rear1_right",
                    "rear2_left", "rear2_right"):
            ax = "front" if "front" in side else ("rear1" if "1" in side else "re
            sd = "left" if "left" in side else "right"
            cfg[f"axle{ax}_{sd}_csv"] = f"{args.data_dir}/{side}.csv"

    evaluate(args.data, args.ckpt, args.norms, args.out,
            cfg_base=cfg, n_plot=args.n_plot, split=args.split)

```